UKRAINIX WATERDRONE

Software Repository and Data Analysis App

Onboarding Guide for New Team Members

UKRAINIX WATERDRONE project

2 November 2026

Contents

1 Introduction **2**
1.1 Project status . . . 2
1.2 Glossary . . . 2

2 Repository structure **3**
2.1 The waterdrone CSV format . . . 3

3 The data analysis app **4**
3.1 General task . . . 4
3.2 Planned features . . . 4
3.3 Design principles . . . 5
3.4 Package layout . . . 6
3.5 Dependency rules . . . 7
3.6 Data flow . . . 7
3.7 How it will look on screen . . . 12
3.8 Adding a physics model . . . 13

4 Test flow **13**
4.1 Phase 1 – synthetic samples . . . 13
4.2 Phase 2 – real-world data . . . 14
4.3 Phase 3 – normalization and extra features . . . 14
4.4 Summary . . . 15

5 Open decisions and where to start **15**
5.1 Open decisions . . . 15
5.2 Suggested starting point . . . 15

1 Introduction

This guide is written for new members of the UKRAINIX WATERDRONE team. It explains

- how the `waterdrone-software` repository is organised (Section 2);
- what the data analysis app is for, which features it will have and how data flows through it (Section 3);
- how the finished app is expected to look on screen (Section 3.7);
- how the app is tested in three phases (Section 4);
- what is still undecided and where a newcomer can start (Section 5).

1.1 Project status

The app does not exist yet. At the time of writing (2 November 2026) the folders of `analysis-app` have been created, each leaf folder holds only a `.gitkeep` placeholder, and no Python script, `__init__.py` or `pyproject.toml` has been written. Everything in this guide therefore describes the agreed *design*. File names, function groups and screens are plans, and every picture that looks like a screenshot is a mock-up drawn with invented data.

An earlier draft of the layout used more folders than the current one. If you see any of the folders `tests`, `config/defaults`, `config/physics_models`, `geo`, `mesh`, `utils`, `state`, `physics/models` or `gui/widgets` inside `analysis-app`, they are leftovers of that draft and are to be deleted (the files with the same names, such as `physics/models.py`, are part of the current plan).

1.2 Glossary

Term	Meaning
Waterdrone	The drone platform that logs its GPS position and concentration-related sensor readings into a CSV file.
Parameter	Any measured quantity, i.e. every CSV column whose name is in capital letters and that is not one of the fixed columns (Section 2.1).
Sample	One parsed CSV file: a list of points with time, local position and one value per parameter.
Local Cartesian coordinates	Position in metres (x, y) relative to a shared reference origin, computed from latitude and longitude.
Control points	A small set of locations that the drone visits quickly, so that the readings there are effectively a snapshot in time.
Normalization sample	The sample made of control points; it serves as the first reference for normalization.

Term	Meaning
Normalization chain	Ordered list of samples in which each sample is normalized against the previous normalized one.
Triangulation (mesh)	Triangles connecting the sample points; the basis for interpolation and isolines.
Isoline	A line connecting positions of equal parameter value.
Origin	A point from which the measured field spreads, found by fitting a physics model to the data.
Physics model	A forward model that computes the field produced by a given set of origins; chosen per parameter in a dictionary text file.

2 Repository structure

The repository `waterdrone-software` is to be published on GitHub. It holds a data analysis app, the test data for that app, and (later) a second app. The layout is:

```
waterdrone-software/
|-- analysis-app/          data analysis app (Python)
|   |-- README.md
|   |-- config/            physics_models.txt, defaults.txt
|   |-- docs/
|   `-- src/analysis_app/  the Python package
|-- samples/               all test data and scripts
|   |-- phase1_synthetic/  from the app's physics model
|   |   `-- scripts/
|   |-- phase2_real/       real-world data
|   |   |-- raw/           open-source data as downloaded
|   |   |-- reduced/       converted to waterdrone CSV
|   |   `-- scripts/
|   `-- phase3_normalization/  (not created yet)
`-- (configurator app)     planned, not created yet
```

Part	Purpose
`analysis-app/`	Reads waterdrone CSV files and turns them into normalized samples, isolines, field origins and an interactive 3D plot. Written as functional Python (no classes). Described in detail in Section 3.
`samples/`	The only place for test data. The analysis app has no `tests/` folder of its own: all testing uses the three sample folders (Section 4).
Configurator app	A second app for configuring the waterdrone itself. It is planned for later; nothing exists yet.

2.1 The waterdrone CSV format

Every sample, synthetic or real, uses the same CSV format.

- `TIMESTAMP` – integer, the computer time at which the row was logged.
- `LATITUDE`, `LONGITUDE` – position in degrees.
- `Z` – measurement depth, when present. It is *not* the vertical axis of the plot (Section 3.6).
- All other columns are parameters. Their names are written in capital letters. Only concentration-related quantities are kept; absolute-value quantities are not part of the format.

The app detects the fixed columns and treats every remaining capitalised column as a parameter, so the number of parameters is not fixed in advance. A made-up example:

```
TIMESTAMP,LATITUDE,LONGITUDE,Z,PARAM_A,PARAM_B
1730500000,50.00010,30.00020,0.5,12.4,0.83
1730500001,50.00011,30.00022,0.5,12.6,0.80
1730500002,50.00013,30.00025,0.5,13.1,0.79
```

3 The data analysis app

3.1 General task

The waterdrone drives over a body of water and logs its position together with concentration-related readings. The analysis app turns such a file into a picture of how each measured quantity is distributed over the water. For every parameter it

1. converts the GPS track into local Cartesian coordinates in metres;
2. optionally *normalizes* the readings against a quick reference survey, so that long surveys or several surveys become comparable;
3. builds a triangulated map of the normalized readings and extracts *isolines* from it;
4. finds the *origins* of the field by least-squares fitting of a physics model;
5. shows readings, isolines and origins in an interactive 3D plot.

3.2 Planned features

- **CSV import with validation.** Fixed columns and capitalised parameter columns are detected automatically. Rows with missing values (NaN), GPS glitches and duplicate timestamps are caught.
- **Coordinate conversion.** Latitude and longitude become local x, y in metres around a shared reference origin, together with an estimate of the projection error.

- **Per-parameter computation.** Everything is computed for every parameter column, including each parameter's own normalization correction. The dropdown in the GUI only selects which result is shown.
- **Chained normalization (optional).** A fast sample of several control points normalizes a big CSV; a still bigger sample can then be normalized against that result, and so on. The first step may be skipped.
- **Triangulation and isolines.** Triangles that bridge gaps or land are dropped; isoline crossings are found on triangle edges and joined by a smoothing spline.
- **Origin finding.** Multi-origin least-squares fit of a physics model, with bounds and diagnostics (RMS error, residual map, a sanity check of the number of origins, comparison with known origins on synthetic data).
- **Pluggable physics models.** The model used for each parameter is named in a dictionary text file. Adding a model means adding one function and one dictionary entry; the solver is not touched.
- **Configurable defaults.** Spline smoothing, isoline levels and similar settings live in a text file, not in the code.
- **Interactive 3D visualization** in matplotlib, with a parameter dropdown and switches for the overlays.

3.3 Design principles

- **Functional Python, no OOP.** Pure functions with data passed explicitly. Data containers are plain `NamedTuple`, `TypedDict` or numpy arrays without behaviour.
- **One folder per concern, one file per group of related functions.** A file is roughly 150–400 lines. A file is split only when it stops being one coherent group; an outgrown file becomes a sub-package behind the same public names.
- **Packages expose a public interface.** Each package's `__init__.py` re-exports its public functions, and other packages import from the package, never from its files. Files can then be reorganised without touching callers.
- **Side effects at the edges.** File reading, GUI and plotting sit at the edges; `pipeline/` composes pure functions.
- **Normalization before triangulation.** Isolines and origin fitting are always built on the *normalized* sample, never on raw data.

3.4 Package layout

The Python package lives in `analysis-app/src/analysis_app/`:

```
src/analysis_app/
|-- main.py              entry point: starts the GUI
|-- core/                types, config reader, functional helpers
|-- parsing/             CSV reading and validation, coordinates
|-- normalization/       matching, correction, chain
|-- isolines/            triangulation, contours
|-- physics/             model dictionary and forward models
|-- inversion/           least-squares origin fitting, diagnostics
|-- pipeline/            stages, run, state
|-- viz/                 3D plot and overlays
`-- gui/                 window, widgets, callbacks
```

The planned files (23 in total) and what each one groups together:

File	Responsibility
`main.py`	Entry point; starts the GUI.
`core/` – **shared foundations**	
`types.py`	Data containers (sample, mesh, isoline, origin, model spec, normalization result), CSV column-name constants, project exceptions.
`config.py`	Reader for the text/dictionary files, access to defaults, resolution of the config and samples paths.
`functional.py`	Compose/pipe helpers, per-parameter mapping, array helpers.
`parsing/` – **from CSV to a sample**	
`csv_parser.py`	Read the CSV, detect fixed columns and capitalised parameters, validate rows (NaN, GPS glitches, duplicate timestamps), build the sample.
`coordinates.py`	Choose the shared reference origin, convert latitude/longitude to local x, y and back, estimate the projection error.
`normalization/` – **runs before isolines**	
`matching.py`	Extract control points, pair points of the big sample with them by distance and time, look up reference values at arbitrary points (spatial index).
`correction.py`	Fit and apply the time-dependence correction; residual check.
`chain.py`	Ordered, skippable chain; promote a normalized sample to be the reference grid of the next one.
`isolines/` – **works on normalized samples**	
`triangulation.py`	Triangulate; drop triangles bridging gaps or land; locate the triangle of a point; interpolate a value at a point.
`contours.py`	Choose levels, find edge crossings, link them into polylines, spline smoothing; isolines for one parameter.
`physics/` – **field models**	

File	Responsibility
`spec.py`	Load the model dictionary, map a CSV parameter to a model, bounds, validation.
`models.py`	Forward models: the field produced by given origins. May become a sub-package later.
`inversion/` – **finding the origins**	
`fit.py`	Initial guess, residuals, least-squares solve for several origins, parameter packing, bounds.
`diagnostics.py`	RMS, residual map, sanity check of the origin count, comparison with known origins (phase 1).
`pipeline/` – **composition**	
`stages.py`	Load, normalize, isoline and origin stages (inputs to result).
`run.py`	Run one or all parameters, result cache, optional parallelism.
`state.py`	Immutable app state plus pure update functions (add a sample, set the chain, select a parameter, store results).
`viz/` – **drawing**	
`plot3d.py`	3D axes, surface/points, labels and style.
`overlays.py`	Draw isolines and origins; toggles.
`gui/` – **user interface**	
`app.py`	Window and layout.
`widgets.py`	CSV picker, normalization-sample picker, parameter dropdown, build/show controls.
`callbacks.py`	Event handlers: update the state, call the pipeline, refresh the plot.

3.5 Dependency rules

Imports may only point downwards in Figure 1; nothing imports upwards. The most important consequences are:

- `normalization` must *not* import `isolines`. It runs first and does its own spatial lookups (`matching.py`).
- `physics` does not depend on the other domain packages, so the synthetic-sample scripts in `samples/phase1_synthetic/scripts/` can import it and generate data with exactly the model the app uses.
- `viz` and `gui` see only results delivered by `pipeline`; they never call the domain packages directly.

3.6 Data flow

Figure 2 shows the complete flow from a CSV file to the plot. Steps 2 to 5 are executed once for every parameter column.

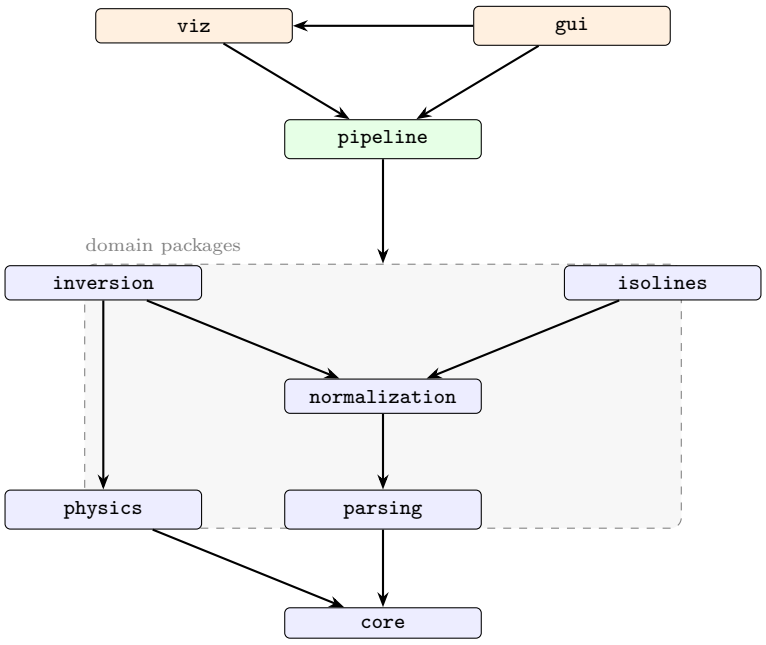


Figure 1: Allowed import directions (arrow = "may import"). Isolines and inversion consume the normalized sample; viz and gui only talk to the pipeline.

Step 1 – Parsing and coordinates

`parsing/csv_parser.py` reads the file, finds the fixed columns and the parameters, and rejects or flags invalid rows. `parsing/coordinates.py` then converts latitude φ and longitude λ into local metres around a shared reference point (φ_0, λ_0). For a small area a simple choice is

$$x = R \cos \varphi_0 \, (\lambda - \lambda_0), \qquad y = R \, (\varphi - \varphi_0),$$

with angles in radians and R the Earth radius. This formula is only an illustration. The projection actually used, and the estimate of its error, follow the team's lecture notes on coordinate systems, geographic coordinate translation and error derivation (topic T3), which are kept outside the repository. The reference origin is shared by all samples so that they overlay correctly.

Step 2 – Normalization (optional)

A large survey takes a long time, so its readings mix spatial variation with changes over time. A short survey of a few control points is nearly a snapshot. Normalization uses such a snapshot as reference (Figure 3):

1. `matching.py` pairs points of the big sample with the reference by distance and by time and looks up reference values at arbitrary points through a spatial index.

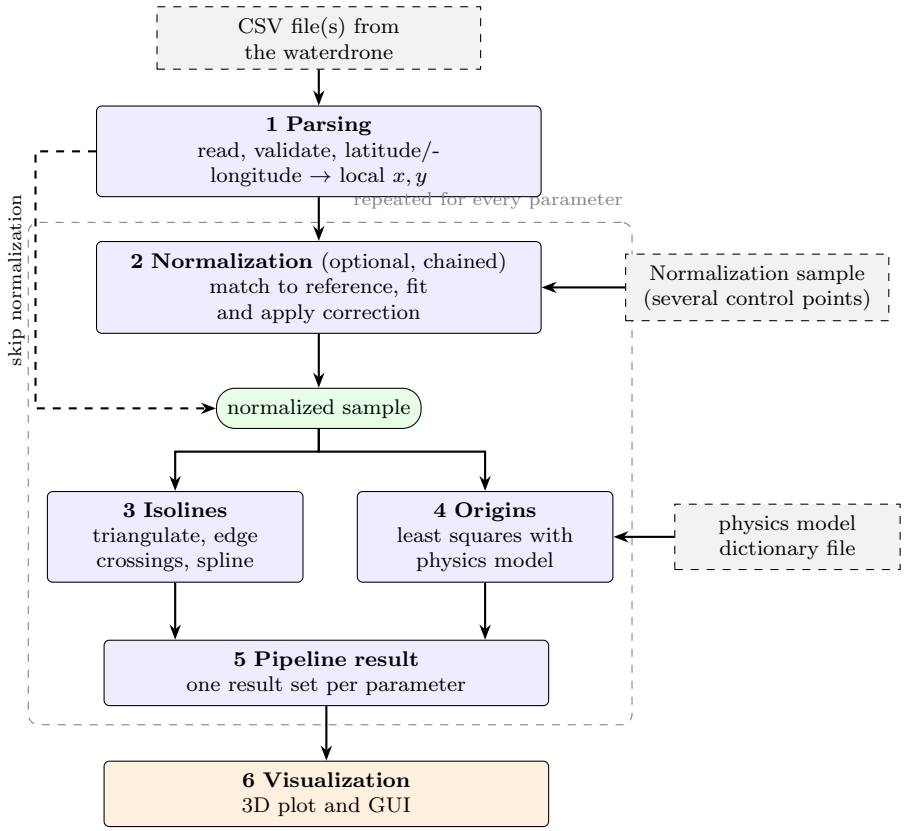


Figure 2: Data flow of the analysis app. Dashed grey boxes are inputs from outside the pipeline; the dashed arrow shows that normalization may be skipped.

2. `correction.py` fits a correction g so that, on the matched pairs, the corrected reading $g(c_i, t_i; \boldsymbol{\phi})$ agrees with the reference value r_i:

$$\min_{\boldsymbol{\phi}} \sum_i \big(g(c_i, t_i; \boldsymbol{\phi}) - r_i\big)^2,$$

   applies it to the whole sample and checks the residuals. The form of g (offset, scale or time drift) is still an open decision (Section 5.1).

3. `chain.py` organises the samples as an ordered chain. A further, bigger sample is normalized against the *previous normalized sample*, which then plays the role of the reference grid. The first step may be skipped when no control-point sample exists.

Each parameter gets its own correction; they are never shared between parameters.

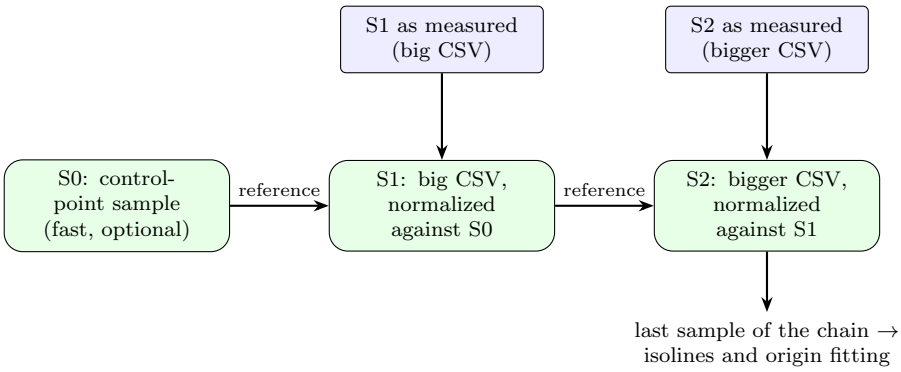


Figure 3: A normalization chain with two links. Each normalized sample becomes the reference grid of the next one; the chain can be longer.

Step 3 – Triangulation and isolines

`isolines/triangulation.py` triangulates the normalized sample and drops triangles that bridge gaps in the track or span land. For each isoline level L, `contours.py` inspects every triangle edge whose end values v_1 and v_2 lie on different sides of L. The crossing lies at

$$\mathbf{p} = \mathbf{p}_1 + t\,(\mathbf{p}_2 - \mathbf{p}_1), \qquad t = \frac{L - v_1}{v_2 - v_1}.$$

The crossings of one triangle are connected by a segment, the segments of neighbouring triangles are linked into polylines, and a spline smooths each polyline (Figure 4). Levels and smoothing come from `defaults.txt` unless the user changes them.

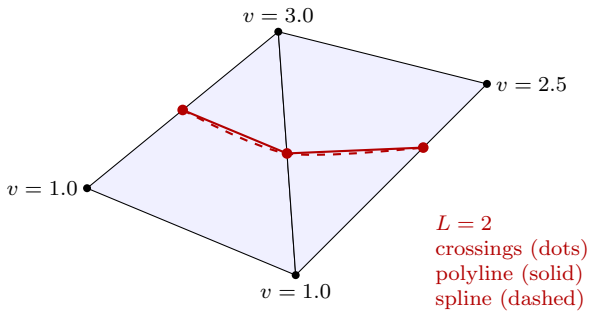


Figure 4: Two neighbouring triangles with vertex values. For the level $L = 2$ the isoline crosses three edges, at $t = 0.5$ on the first two and $t \approx 0.67$ on the third; the crossings are linked and smoothed.

Step 4 – Origin finding

`physics/spec.py` reads the dictionary text file and tells the solver which physics model belongs to which CSV parameter, with bounds. `physics/models.py` implements the forward models. For a model f and a set of K origins described by parameters $\boldsymbol{\theta}$, the solver in `inversion/fit.py` minimizes the sum of squared residuals over the normalized sample points (x_i, y_i) with values c_i:

$$\min_{\boldsymbol{\theta}} \sum_i \big(c_i - f(x_i, y_i; \boldsymbol{\theta})\big)^2 \quad \text{subject to the bounds from the dictionary.}$$

`inversion/diagnostics.py` then reports the RMS error, a residual map and a sanity check of the number of origins. For synthetic samples it also compares the result with the known origins.

An illustrative (not final) dictionary file could look like this; the real format is fixed when `physics/spec.py` is written.

```
# PARAMETER   MODEL
PARAM_A       model_one
PARAM_B       model_two
```

Step 5 and 6 – Pipeline and visualization

`pipeline/stages.py` wraps steps 1–4 as stages, `run.py` runs them for one or all parameters and caches the results, and `state.py` keeps the immutable application state. `viz/plot3d.py` draws a matplotlib 3D plot whose horizontal axes are the local coordinates x, y in metres and whose vertical axis is the *parameter value*. (The CSV column `Z` is measurement depth and is not used as the vertical axis.) `viz/overlays.py` adds isolines and origins on request.

3.7 How it will look on screen

The GUI is a single window; the toolkit is likely Tkinter and is isolated in `gui/` so it can be exchanged. Figure 5 is a mock-up with invented data and file names.

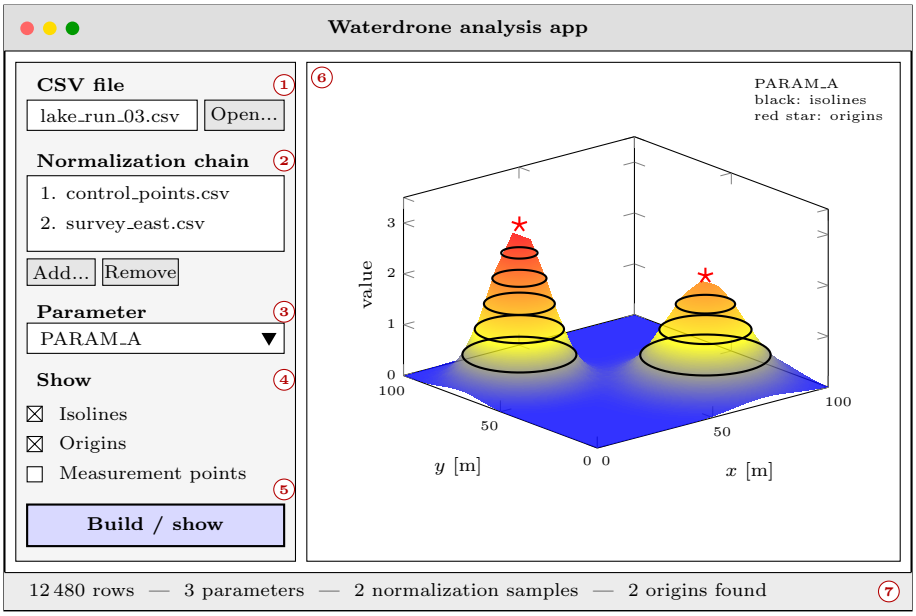


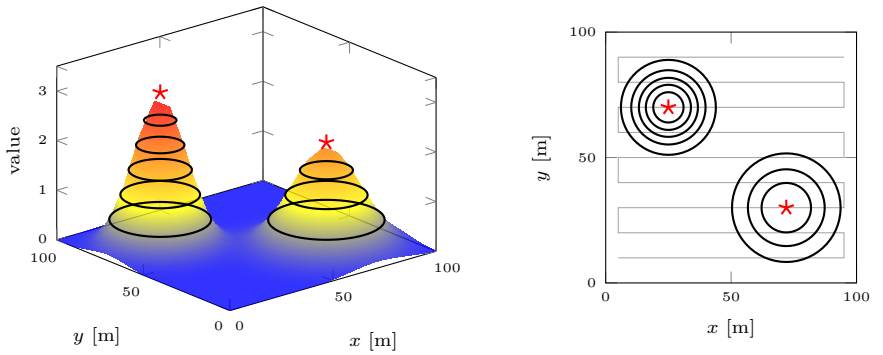
Figure 5: Mock-up of the main window with invented data: (1) CSV picker, (2) normalization chain, (3) parameter dropdown, (4) display toggles, (5) build button, (6) 3D plot, (7) status bar.

A typical session is expected to run as follows.

1. The user picks a CSV file with the CSV picker (1). It is parsed and converted to local coordinates immediately.
2. Optionally, the user adds a control-point sample and further samples to the normalization list (2). The order in the list is the order of the chain.
3. The user selects a parameter in the dropdown (3) and presses *Build / show* (5). The pipeline computes all parameters; the selected one is drawn.
4. Isolines and origins can be toggled (4). Switching the dropdown shows another parameter without recomputation, because all parameters were already computed (or are cached).
5. The 3D plot (6) can be rotated and zoomed with the usual matplotlib tools; the status bar (7) reports row counts and what was found.

Figure 6 shows what the plot is meant to convey: the field as a surface over the local x, y plane, isolines at fixed levels, the origins the least-squares fit found,

and, in the top view, the drone track that produced the data. The field in this picture is two invented smooth hills, not a measurement.



(a) 3D view

(b) top view with drone track (grey)

Figure 6: Possible render of one parameter (synthetic field): isolines in black, fitted origins as red stars.

3.8 Adding a physics model

Because the solver reads the model from the dictionary, a new model needs only two edits:

1. add a forward-model function to `physics/models.py`;
2. add a dictionary entry that maps a parameter to the new model name (and its bounds).

The synthetic-sample scripts of phase 1 use the same function, so a new model can be checked on generated data straight away.

4 Test flow

The analysis app has no `tests/` folder. All testing is done with the repository-level `samples/` folder, in three phases that grow in difficulty (Figure 7). Because `pipeline/` contains no GUI code, a script in `samples/` can run a sample through the same stages the GUI uses.

4.1 Phase 1 – synthetic samples

Folder: `samples/phase1_synthetic/` (generator scripts in `scripts/`).

Idea. The samples are *abstract*: they are generated with the same physics model that is pre-configured in the app. A script chooses origins with known positions, evaluates the forward model from `analysis_app.physics` along a

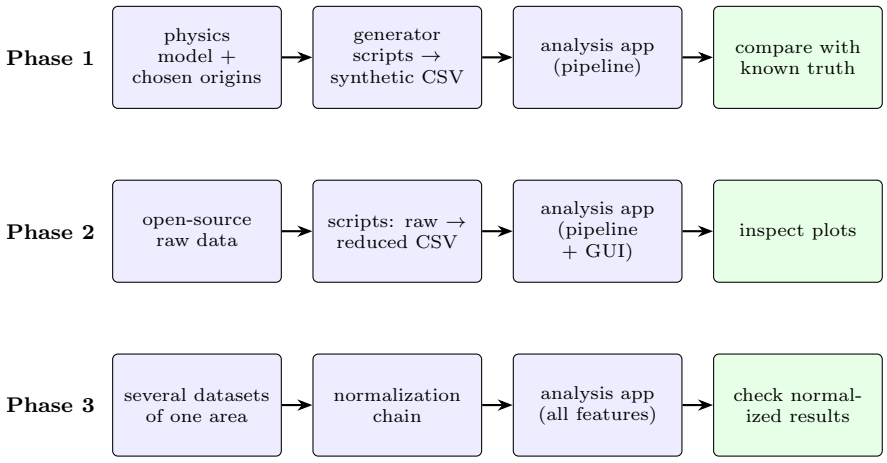


Figure 7: The three test phases. Each lane shows where the data comes from and what is checked at the end.

track, and writes a CSV file in the waterdrone format. The truth is therefore known exactly.

What it checks. The *correctness* of the app. For example: parsing and coordinate conversion, whether the isolines lie where the model says they should, and above all whether the least-squares fit recovers the known origins (`inversion/diagnostics.py` compares the result with them). Variants such as gaps in the track can be generated to exercise the handling of triangles that bridge gaps. Acceptance thresholds are still to be agreed.

4.2 Phase 2 – real-world data

Folder: `samples/phase2_real/` with three subfolders:

- `raw/` – open-source data as downloaded (the Utah Lake data is the first data set);
- `reduced/` – the same data converted to the waterdrone CSV format, keeping only concentration-related parameters;
- `scripts/` – the conversion scripts from `raw/` to `reduced/`.

What it checks. The *visualization* on real data: how the 3D plot, isolines and overlays behave with real tracks, noise and irregular coverage. There are no known origins here, so quality is judged by inspecting the plots.

4.3 Phase 3 – normalization and extra features

Folder: `samples/phase3_normalization/` – not created yet.

Idea. The ultimate test. It uses the extra features, above all the normalization of data with the help of several different data sets, so the whole normalization chain (Figure 3) is exercised end to end, and the normalized result then feeds isolines and origin fitting.

What it checks. That the app works with several data sets of one area at once. The data sets and acceptance criteria are still to be chosen.

4.4 Summary

Phase	Folder	Data	Main question
1	`samples/phase1_synthetic/`	generated from the app's physics model	Is the app correct?
2	`samples/phase2_real/`	open-source real data, reduced to the waterdrone format	Does the visualization work on real data?
3	`samples/phase3_normalization/` (to be created)	several data sets of one area	Does normalization across data sets work?

5 Open decisions and where to start

5.1 Open decisions

- **Normalization correction model.** Offset, scale or time drift? This must be decided before `normalization/correction.py` is written.
- **Which sample feeds isolines and origin fitting.** The assumption is the last sample of the chain; the alternative is all samples merged.
- **GUI toolkit.** Tkinter is the likely default; the choice is isolated in `gui/`.
- **Physics model format and the first models.** The dictionary file format and the concrete models are not fixed yet.
- **Phase 3 data sets and acceptance criteria**, as well as the thresholds for phases 1 and 2.

5.2 Suggested starting point

Following the dependency direction of Figure 1, a natural order for the first implementation is: `core`, then `parsing` and `physics` (so that phase-1 samples can be generated early), then `isolines`, `inversion` and `normalization`, then `pipeline`, and finally `viz` and `gui`. Whatever you pick up, keep to the rules of Section 3.3: functions instead of classes, one file per group of related functions, imports only through package interfaces.

The second app, which configures the waterdrone, is planned for later and has no structure yet.